

Q1 Try-Catch-Finally Basics

25 marks

Q1

Try-Catch-Finally Basics
25 marks

Q2

Producer-Consumer with
BlockingQueue
25 marks

Q3

Q4 

wait() and notify() Coordination

25 marks

4/4 answered

```
public class TryCatchFinallyDemo {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};

        try {
            System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException expected here
        } catch (NullPointerException e) { // Bug 1: wrong exception type caught, real exception not handled
            System.out.println("Caught null pointer");
        }

        try {
            int value = Integer.parseInt("abc");
        } catch (Exception e) {
        }

        String str = null;

        try {
            System.out.println(str.length());
        } catch (NullPointerException e) {
            return;
        } finally {
            System.out.println("Cleanup");
            str.length();
        }

        try {
            riskyMethod();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    static void riskyMethod() throws java.io.IOException {
        throw new java.io.IOException("failure");
    }
}
```



SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA

192424064RA

▼

Your Code

```
1 import java.io.IOException;
2 public class TryCatchFinallyDemo {
3     public static void main(String[] args) {
4         int[] numbers = {1, 2, 3};
5         try {
6             System.out.println(numbers[5]);
7         } catch (ArrayIndexOutOfBoundsException e) {
8             System.out.println("Array index out of bounds");
9         }
10        try {
11            int value = Integer.parseInt("abc");
12            System.out.println(value);
13        } catch (NumberFormatException e) {
14            System.out.println("Invalid number format");
15        }
16        String str = null;
17        try {
18            System.out.println(str.length());
19        } catch (NullPointerException e) {
20            System.out.println("String is null");
21        } finally {
22            System.out.println("Cleanup");
23        }
24        try {
25            riskyMethod();
26        } catch (IOException e) {
27            System.out.println("IOException: " + e.getMessage());
28        }
29    }
30    static void riskyMethod() throws IOException {
31        throw new IOException("failure");
32    }
33 }
```



Run



Save

Input

stdin input

Output

```
Array index out of bounds
Invalid number format
String is null
Cleanup
IOException: failure
```




SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA
192424064RA

CO3-AT1-Debugging

← Back

QUESTIONS

- Q1

Try-Catch-Finally Basics

25 marks

✓
- Q2

Producer-Consumer with BlockingQueue

25 marks

✓
- Q3

Thread Join and Interruption

25 marks

✓
- Q4

wait() and notify() Coordination

25 marks

✓

4/4 answered

Q2 Producer-Consumer with BlockingQueue

25 marks

```
import java.util.concurrent.*;

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        Thread producer = new Thread(() -> {
            for (int i = 0; i < 10; i++) {
                queue.add(i);
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 0; i < 10; i++) {
                int val = queue.poll();
                System.out.println("Consumed: " + val);
            }
        });

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("Queue size: " + queue.size());
    }
}
```

Your Code

```
1 import java.util.concurrent.*;
2 public class ProducerConsumerDemo {
3     public static void main(String[] args) throws InterruptedException
```

Input

stdin input



SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA
192424064RA

```
}
```

Your Code

```
1 import java.util.concurrent.*;
2 public class ProducerConsumerDemo {
3     public static void main(String[] args) throws InterruptedException
4     {
5         BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
6         Thread producer = new Thread(() -> {
7             try {
8                 for (int i = 0; i < 10; i++) {
9                     queue.put(i);
10                    System.out.println("Produced: " + i);
11                }
12            } catch (InterruptedException e) {
13                Thread.currentThread().interrupt();
14            }
15        });
16        Thread consumer = new Thread(() -> {
17            try {
18                for (int i = 0; i < 10; i++) {
19                    int val = queue.take();
20                    System.out.println("Consumed: " + val);
21                }
22            } catch (InterruptedException e) {
23                Thread.currentThread().interrupt();
24            }
25        });
26        producer.start();
27        consumer.start();
28        producer.join();
29        consumer.join();
30        System.out.println("Queue size: " + queue.size());
31    }
```

▶ Run

☁ Save

Input

stdin input

Output

```
Consumed: 0
Produced: 0
Produced: 1
Consumed: 1
Consumed: 2
Produced: 2
```

Output

Produced: 2

Consumed: 3

Produced: 3

Produced: 4

Consumed: 4

Produced: 5

Output

Produced: 6

Consumed: 5

Consumed: 6

Produced: 7

Consumed: 7

Consumed: 8

Output

Consumed: 7

Consumed: 8

Produced: 8

Produced: 9

Consumed: 9

Queue size: 0



CO3-AT1-Debugging

← Back

QUESTIONS

- Q1

Try-Catch-Finally Basics

25 marks
- Q2

Producer-Consumer with BlockingQueue

25 marks
- Q3

Thread Join and Interruption

25 marks
- Q4

wait() and notify() Coordination

25 marks

4/4 answered

Q3 Thread Join and Interruption

25 marks

```
public class ThreadJoinDemo {
    public static void main(String[] args) {
        Thread worker = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("Working: " + i);
            }
            Thread.sleep(500);
        } catch (InterruptedException e) {
            // Bug 1: interrupted exception caught but loop continues, interrupt flag not restored or checked
        }
    }
});

worker.start();
worker.interrupt();

worker.join(1000)

try {
    worker.join(1000);
} catch (Exception e) {
}

if (worker.isAlive) {
    System.out.println("Still running");
}

Thread.currentThread.interrupt();
}
```

Your Code

Input



SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA
192424064RA

Your Code

```
1 public class ThreadJoinDemo {
2     public static void main(String[] args) {
3         Thread worker = new Thread(() -> {
4             for (int i = 0; i < 5; i++) {
5                 System.out.println("Working: " + i);
6                 try {
7                     Thread.sleep(500);
8                 } catch (InterruptedException e) {
9                     System.out.println("Worker interrupted");
10                    Thread.currentThread().interrupt();
11                    break;
12                }
13            }
14        });
15        worker.start();
16        worker.interrupt();
17        try {
18            worker.join(1000);
19        } catch (InterruptedException e) {
20            System.out.println("Main thread interrupted");
21            Thread.currentThread().interrupt();
22        }
23        if (worker.isAlive()) {
24            System.out.println("Still running");
25        } else {
26            System.out.println("Worker finished");
27        }
28    }
29 }
```

▶ Run

☁ Save

Input

stdin input

Output

```
Working: 0
Worker interrupted
Worker finished
```

CO3-AT1-Debugging

← Back

QUESTIONS

Q1

Try-Catch-Finally Basics

25 marks

✓

Q2

Producer-Consumer with BlockingQueue

25 marks

✓

Q3

Thread Join and Interruption

25 marks

✓

Q4

wait() and notify() Coordination

25 marks

✓

4/4 answered

Q4

wait() and notify() Coordination

25 marks

```
public class WaitNotifyDemo {
    private boolean dataReady = false;
    private final Object lock = new Object();

    public void produce() {
        synchronized (lock) {
            dataReady = true;
            System.out.println("Data produced");
            lock.notify;
        }
    }

    public void consume() {
        if (!dataReady) {
            try {
                lock.wait();
            } catch (InterruptedException e) {
            }
        }
        System.out.println("Data consumed");
    }

    public void consumeCorrectly() {
        synchronized (lock) {
            if (!dataReady) {
                try {
                    lock.wait();
                } catch (InterruptedException e) {
                    Thread.interrupted();
                }
            }
        }
    }
}
```




SIMATS Online Programming Lab

JAVA PROGRAMMING

M MOSHIKA

192424064RA

⌵

```
public static void main(String[] args) {
    WaitNotifyDemo demo = new WaitNotifyDemo();
    Thread producer = new Thread(demo::produce);
    Thread consumer = new Thread(demo::consumeCorrectly);
    consumer.start();
    producer.start();
    consumer.join();
}
```

Your Code

```
1 public class WaitNotifyDemo {
2     private boolean dataReady = false;
3     private final Object lock = new Object();
4     public void produce() {
5         synchronized (lock) {
6             dataReady = true;
7             System.out.println("Data produced");
8             lock.notify();
9         }
10    }
11    public void consumeCorrectly() {
12        synchronized (lock) {
13            while (!dataReady) {
14                try {
15                    lock.wait();
16                } catch (InterruptedException e) {
17                    Thread.currentThread().interrupt();
18                    return;
19                }
20            }
21            System.out.println("Data consumed");
22        }
23    }
24    public static void main(String[] args) {
25        WaitNotifyDemo demo = new WaitNotifyDemo();
26        Thread producer = new Thread(demo::produce);
27        Thread consumer = new Thread(demo::consumeCorrectly);
28        consumer.start();
```

Input

stdin input

Output

```
Data produced
Data consumed
```



M MOSHIKA
192424064RA

Output

▶ Run

 Save